

TX32F01 Cortex-M0 内存分配与外设寄存器地址图

Codex 生成，依据工程头文件与当前 Bootloader 布局整理

2026 年 5 月 21 日

1 基本地址计算模型

TX32F01 的外设采用 memory-mapped I/O。CPU 访问外设寄存器时，本质上是在访问某个固定地址。对任意外设 P ，若其基地址为 B_P ，寄存器偏移为 O_R ，则寄存器绝对地址为

$$A(P, R) = B_P + O_R$$

例如 UART 基地址为

$$B_{\text{UART}} = 0\text{x}49002000$$

UART 发送数据寄存器 TDR 的偏移为

$$O_{\text{TDR}} = 0\text{x}10$$

因此：

$$A(\text{UART}, \text{TDR}) = 0\text{x}49002000 + 0\text{x}10 = 0\text{x}49002010$$

对位域访问，若寄存器值为 V ，字段起始 bit 为 s ，字段宽度为 w ，则字段 mask 为

$$M(s, w) = (2^w - 1) \ll s$$

读取字段值：

$$F = \frac{V \& M(s, w)}{2^s}$$

写入字段值 x ：

$$V' = (V \& \sim M(s, w)) \mid ((x \ll s) \& M(s, w))$$

例如 UART CER.UE 位于 bit 0，宽度 1：

$$M_{\text{UE}} = (2^1 - 1) \ll 0 = 0\text{x}00000001$$

置位 UART 使能：

$$\text{UART} \rightarrow \text{CER} \leftarrow \text{UART} \rightarrow \text{CER} \mid 0\text{x}00000001$$

2 总体地址空间

地址范围或基址	区域	说明
0x00000000	Code alias / 启动映射区	复位后 Cortex-M0 从这里取初始 MSP 和 ResetHandler。实际内容映射到片内 Flash 起始区域。
0x01000000--0x01007FFF	Flash, 32 KB	存放 Bootloader、APP、meta、boot flag。
0x20000000--0x20000FFF	SRAM, 4 KB	存放软向量表、全局变量、栈、RTOS task stack、临时 buffer。
0x40001000	SCU	系统控制、时钟、外设使能、复位、低功耗。
0x40008000	FLASH 控制器	控制 Flash 擦写、CRC、保护、Die ID。注意这不是代码存储地址。
0x49000000--0x49007FFF	外设寄存器区	TIM、SPI、GPIO、UART、I2C、ADC、IWDG、EXTI。
0xE000E000 起	Cortex-M0 PPB	SysTick、NVIC、SCB 等 ARM 内核系统外设。

3 Flash 分区计算

片内 Flash 总容量：

$$S_{\text{flash}} = 32 \text{ KB} = 32 \times 1024 = 0x8000$$

Flash 起止地址：

$$F_{\text{base}} = 0x01000000$$

$$F_{\text{end}} = F_{\text{base}} + S_{\text{flash}} - 1 = 0x01000000 + 0x8000 - 1 = 0x01007FFF$$

当前工程采用如下分区：

起始地址	结束地址	大小	用途
0x01000000	0x01001FFF	8 KB	Bootloader：向量表、YMODEM、Flash 写入、CRC、跳转 APP。
0x01002000	0x010077FF	22 KB	Application：APP/APP_COOP/APP_LOG/APP_IRQ 等。
0x01007800	0x01007BFF	1 KB	APP meta：magic、app_size、crc16。
0x01007C00	0x01007FFF	1 KB	Boot flag：如 0xA55AF00D 表示强制留在 Bootloader。

Bootloader 大小：

$$S_{\text{BL}} = 8 \text{ KB} = 0x2000$$

$$F_{\text{BL,end}} = 0x01000000 + 0x2000 - 1 = 0x01001FFF$$

APP 起始地址:

$$F_{\text{APP,base}} = F_{\text{base}} + S_{\text{BL}} = 0\text{x}01000000 + 0\text{x}2000 = 0\text{x}01002000$$

APP 最大容量:

$$S_{\text{APP}} = 0\text{x}01007800 - 0\text{x}01002000 = 0\text{x}5800 = 22 \text{ KB}$$

因此, OTA 接收 APP 镜像时必须满足:

$$0 < \text{app_size} \leq 0\text{x}5800$$

写入地址范围必须满足:

$$0\text{x}01002000 \leq A_{\text{write}} \leq 0\text{x}010077FF$$

meta 区地址:

$$F_{\text{meta,base}} = 0\text{x}01007800$$

$$F_{\text{meta,end}} = 0\text{x}01007BFF$$

boot flag 区地址:

$$F_{\text{flag,base}} = 0\text{x}01007C00$$

$$F_{\text{flag,end}} = 0\text{x}01007FFF$$

4 SRAM 分配计算

SRAM 总容量:

$$S_{\text{sram}} = 4 \text{ KB} = 0\text{x}1000$$

SRAM 起止地址:

$$R_{\text{base}} = 0\text{x}20000000$$

$$R_{\text{end}} = R_{\text{base}} + S_{\text{sram}} - 1 = 0\text{x}20000000 + 0\text{x}1000 - 1 = 0\text{x}20000FFF$$

因为 Cortex-M0 没有 VTOR, 当前 Bootloader 设计在 SRAM 头部保留软向量表。软向量表大小为 96 B:

$$S_{\text{softvec}} = 96 = 0\text{x}60$$

软向量表范围:

$$R_{\text{softvec,base}} = 0\text{x}20000000$$

$$R_{\text{softvec,end}} = 0\text{x}20000000 + 0\text{x}60 - 1 = 0\text{x}2000005F$$

APP 可用 RAM 起始地址必须避开软向量表:

$$R_{\text{APP,base}} = R_{\text{softvec,end}} + 1 = 0\text{x}20000060$$

APP 最大可用 SRAM:

$$S_{\text{APP,RAM}} = 0\text{x}20001000 - 0\text{x}20000060 = 0\text{x}FA0 = 4000 \text{ B}$$

实际 APP 还必须在这 4000 B 中同时容纳：

$$S_{RW} + S_{ZI} + S_{\text{heap}} + S_{\text{MSP}} + \sum_i S_{\text{PSP},i} \leq 4000 \text{ B}$$

裸机或协作式调度器通常只有一个主栈：

$$S_{RW} + S_{ZI} + S_{\text{buffer}} + S_{\text{MSP}} \leq 4000 \text{ B}$$

FreeRTOS 需要额外考虑每个任务的独立栈：

$$S_{\text{RTOS}} = S_{\text{kernel}} + S_{\text{idle_stack}} + \sum_{i=1}^N S_{\text{task},i} + S_{\text{queues}} + S_{\text{timers}}$$

5 外设基地址与地址公式

模块	基地址	地址计算
SCU	0x40001000	$A = 0x40001000 + O$
FLASH 控制 器	0x40008000	$A = 0x40008000 + O$
TIM0	0x49000000	$A = 0x49000000 + O$
TIM1	0x49000400	$A = 0x49000400 + O$
TIM2	0x49000800	$A = 0x49000800 + O$
SPI	0x49000C00	$A = 0x49000C00 + O$
GPIO0	0x49001000	$A = 0x49001000 + O$
GPIO1	0x49001400	$A = 0x49001400 + O$
GPIO2	0x49001800	$A = 0x49001800 + O$
GPIO3	0x49001C00	$A = 0x49001C00 + O$
UART	0x49002000	$A = 0x49002000 + O$
I2C	0x49005000	$A = 0x49005000 + O$
ADC	0x49007000	$A = 0x49007000 + O$
IWDG	0x49009000	$A = 0x49009000 + O$
EXTI	0x4900B000	$A = 0x4900B000 + O$

Timer 和 GPIO 都是重复实例。Timer 的第 n 个实例可以写为：

$$B_{\text{TIM}n} = 0x49000000 + n \times 0x400, \quad n \in \{0, 1, 2\}$$

GPIO 的第 n 个实例可以写为：

$$B_{\text{GPIO}n} = 0x49001000 + n \times 0x400, \quad n \in \{0, 1, 2, 3\}$$

例如 GPIO3 的输出数据寄存器 ODR 偏移为 0x28：

$$A(\text{GPIO3}, \text{ODR}) = (0x49001000 + 3 \times 0x400) + 0x28 = 0x49001C28$$

6 SCU 寄存器

SCU 基地址:

$$B_{\text{SCU}} = 0x40001000$$

偏移	绝对地址	寄存器	说明
0x00	0x40001000	PKR	访问保护 key。
0x04	0x40001004	CCR	HCLK 分频、MCO 输出。
0x08	0x40001008	PENR	外设时钟使能。
0x0C	0x4000100C	PRSTR	外设复位。
0x10	0x40001010	RSR	复位原因。
0x14	0x40001014	PMCR	BOR/PVD 电源监控。
0x18	0x40001018	LPCR	低功耗配置。
0x1C	0x4000101C	DBGCR	debug sleep/stop、Timer/IWDT 调试暂停。
0x20	0x40001020	STR	SysTick 校准值。
0x24	0x40001024	BSR	Boot 映射选择。

外设时钟使能的一般形式:

$$\text{SCU} \rightarrow \text{PENR} \leftarrow \text{SCU} \rightarrow \text{PENR} | (1 \ll k)$$

其中 k 为对应外设的 enable bit, 例如 UART 的时钟使能位为 bit 3:

$$\text{SCU} \rightarrow \text{PENR} \leftarrow \text{SCU} \rightarrow \text{PENR} | (1 \ll 3)$$

7 FLASH 控制器寄存器

FLASH 控制器基地址:

$$B_{\text{FLASHCTL}} = 0x40008000$$

偏移	绝对地址	寄存器	说明
0x00	0x40008000	ASR	Flash 访问状态。
0x04	0x40008004	ACR	访问控制、wait cycle。
0x08	0x40008008	ACHKR	擦写校验使能。
0x0C	0x4000800C	PEKEYR	Program/erase key。
0x20	0x40008020	IFR	Flash 状态/中断标志。
0x24	0x40008024	IER	Flash 中断使能。
0x30	0x40008030	TKEYR	timing 配置 key。
0x34	0x40008034	TNVS	Flash timing。
0x38	0x40008038	TPROG	Program timing。

偏移	绝对地址	寄存器	说明
0x3C	0x4000803C	TADS	Address setup timing。
0x40	0x40008040	TADH	Address hold timing。
0x44	0x40008044	TPGS	Program setup timing。
0x48	0x40008048	TPGH	Program hold timing。
0x4C	0x4000804C	TPRCV	Program recovery timing。
0x50	0x40008050	TSRCV	Sector erase recovery timing。
0x54	0x40008054	TCRCV	Chip erase recovery timing。
0x58	0x40008058	TSERASE	Sector erase timing。
0x5C	0x4000805C	TCERASE	Chip erase timing。
0x60	0x40008060	TRW	Read/write timing。
0xE0	0x400080E0	DIEID0	Die ID 0。
0xE4	0x400080E4	DIEID1	Die ID 1。
0xE8	0x400080E8	DIEID2	Die ID 2。
0xEC	0x400080EC	DIEID3	Die ID 3。
0xF0	0x400080F0	CRCCR	CRC 控制。
0xF4	0x400080F4	CRCARL	CRC 地址低寄存器。
0xF8	0x400080F8	CRCARH	CRC 地址高寄存器。
0xFC	0x400080FC	CRCVR	CRC 结果。
0x100	0x40008100	PKEYR	控制寄存器访问保护 key。
0x104	0x40008104	PRGKEYR	Main Flash program enable key。
0x10C	0x4000810C	NVR_WR_EN	NVR 写使能。

Flash CRC 地址范围要特别注意。若硬件 API 采用 inclusive end, 则对长度为 N 字节的 APP:

$$A_{\text{crc,start}} = F_{\text{APP,base}}$$

$$A_{\text{crc,end}} = F_{\text{APP,base}} + N - 1$$

而不是 $F_{\text{APP,base}} + N$ 。

8 Timer 寄存器

Timer 基地址:

$$B_{\text{TIM}n} = 0x49000000 + n \times 0x400$$

偏移	寄存器	说明
0x00	CR	计数器使能、运行模式、预装载。
0x04	CCCR	Capture/Compare 控制。
0x08	BCR	Break 控制。
0x0C	TCR	Trigger 控制。

偏移	寄存器	说明
0x10	IER	中断使能。
0x14	IFR	中断标志。
0x18	CNT	当前计数值。
0x1C	DIV	分频。
0x20	ARR	自动重装载值。
0x24	CCR	Compare/Capture 值。
0x28	DTG	死区时间。

PWM 频率公式可以写为:

$$f_{\text{PWM}} = \frac{f_{\text{PCLK}}}{(DIV + 1)(ARR + 1)}$$

若系统时钟为 24 MHz, 分频寄存器选择 $DIV = 23$, 自动重装载 $ARR = 999$:

$$f_{\text{PWM}} = \frac{24\,000\,000}{(23 + 1)(999 + 1)} = 1000 \text{ Hz}$$

PWM 占空比近似为:

$$D \approx \frac{CCR}{ARR + 1}$$

9 GPIO 寄存器

GPIO 基地址:

$$B_{\text{GPIO}n} = 0x49001000 + n \times 0x400$$

偏移	寄存器	说明
0x00	MDR	GPIO 模式。
0x04	PUR	上拉。
0x08	PDR	下拉。
0x0C	DSR	驱动强度。
0x10	SRR	Slew rate。
0x14	OTR	推挽/开漏。
0x18	IER	输入使能。
0x20	AFR	复用功能选择。
0x28	ODR	输出数据。
0x2C	BSR	置位输出。
0x30	BRR	清零输出。
0x34	BFR	翻转输出。
0x38	IDR	输入数据。

对 GPIO 第 p 个 pin, 置位输出通常使用:

$$\text{GPIOx-BSR} \leftarrow 1 \ll p$$

清零输出：

$$\text{GPIOx} \rightarrow \text{BRR} \leftarrow 1 \ll p$$

翻转输出：

$$\text{GPIOx} \rightarrow \text{BFR} \leftarrow 1 \ll p$$

相对于读改写 ODR，BSR/BRR/BFR 更适合中断和并发场景。

10 UART 寄存器

UART 基地址：

$$B_{\text{UART}} = 0x49002000$$

偏移	绝对地址	寄存器	说明
0x00	0x49002000	CR	校验、字长、停止位、极性。
0x04	0x49002004	BRR	波特率分频。
0x08	0x49002008	IER	中断使能。
0x0C	0x4900200C	ISR	状态/中断标志。
0x10	0x49002010	TDR	发送数据。
0x14	0x49002014	RDR	接收数据。
0x20	0x49002020	CER	UART/TX/RX 使能。
0x100	0x49002100	SADJ	stop bit 宽度微调。

UART 发送寄存器地址计算：

$$A_{\text{UART,TDR}} = 0x49002000 + 0x10 = 0x49002010$$

UART 接收寄存器地址计算：

$$A_{\text{UART,RDR}} = 0x49002000 + 0x14 = 0x49002014$$

当前工程常用 UART 配置为 115200 8N1。若发送一个字节包含 1 start、8 data、1 stop，则一字节线路时间为：

$$t_{\text{byte}} = \frac{10}{115200} \approx 86.8 \mu s$$

这说明 polling UART 单字节发送可以接受，但长字符串打印会明显占用 CPU：

$$t_{\text{print}} \approx N_{\text{bytes}} \times 86.8 \mu s$$

11 SPI、I2C、ADC、IWDG、EXTI 寄存器摘要

11.1 SPI

$$B_{\text{SPI}} = 0x49000C00$$

偏移	寄存器	说明
0x00	CR	SPI 使能、主从、CPOL/CPHA、NSS、FIFO 清除。
0x04	SR	传输状态、FIFO 状态、错误标志。
0x08	IER	SPI 中断使能。
0x0C	DIVR	SPI 分频。
0x10	DATAR	SPI 数据。
0x14	TCNTR	TX FIFO 计数。
0x18	RCNTR	RX FIFO 计数。

11.2 I2C

$$B_{I2C} = 0x49005000$$

偏移	寄存器	说明
0x00	DR	收发数据。
0x04	AR	从机地址、地址 mask、General Call。
0x08	CR	START、STOP、ACK、使能、控制码。
0x0C	SR	I2C 状态码。

11.3 ADC

$$B_{ADC} = 0x49007000$$

偏移	绝对地址	寄存器	说明
0x00	0x49007000	IFR	ADC 中断标志。
0x04	0x49007004	IER	ADC 中断使能。
0x08	0x49007008	CR	ADC 启动/使能。
0x0C	0x4900700C	CFGR	clock、trigger、VREF。
0x10	0x49007010	SQR1	序列控制 1。
0x14	0x49007014	SQR2	序列控制 2。
0x20	0x49007020	DR1	序列 1 结果。
0x24	0x49007024	DR2	序列 2 结果。
0x28	0x49007028	DR3	序列 3 结果。
0x2C	0x4900702C	DR4	序列 4 结果。
0x30	0x49007030	DR5	序列 5 结果。
0x34	0x49007034	DR6	序列 6 结果。
0x38	0x49007038	DR7	序列 7 结果。
0x3C	0x4900703C	DR8	序列 8 结果。

偏移	绝对地址	寄存器	说明
0x40	0x49007040	OFR	offset。
0x44	0x49007044	VREFRL	2.048 V 内部参考实测值。
0x48	0x49007048	VREFRH	高参考实测值。

若 ADC 为 N bit，原始码值为 C ，参考电压为 V_{ref} ，则输入电压近似为：

$$V_{\text{in}} \approx \frac{C}{2^N - 1} V_{\text{ref}}$$

若用内部参考反推 VDD，内部参考实际电压为 V_{intref} ，ADC 测得码值为 C_{intref} ：

$$V_{\text{DD}} \approx \frac{(2^N - 1)V_{\text{intref}}}{C_{\text{intref}}}$$

11.4 IWDT

$$B_{\text{IWDT}} = 0x49009000$$

偏移	寄存器	说明
0x00	KR	看门狗 key。
0x04	PR	预分频。
0x08	RLR	重装载值。
0x0C	SR	状态。
0x10	IER	中断使能。
0x14	IFR	中断标志。

11.5 EXTI

$$B_{\text{EXTI}} = 0x4900B000$$

偏移	寄存器	说明
0x00	IMR	中断 mask。
0x04	EMR	事件 mask。
0x08	RTSR	上升沿触发选择。
0x0C	FTSR	下降沿触发选择。
0x10	SWIER	软件触发。
0x14	PR	pending 标志。
0x18	CFGR	EXTI line 到 GPIO port 映射。

12 Cortex-M0 系统外设区

Cortex-M0 标准 PPB 区域常用地址如下：

模块	地址	说明
SysTick	0xE000E010	CTRL、LOAD、VAL、CALIB。
NVIC	0xE000E100	IRQ enable、disable、pending、priority。
SCB	0xE000ED00	CPUID、ICSR、AIRCRR、SCR、CCR、SHPR、SHCSR。

当前工程特别要注意：

Cortex-M0 has no VTOR

因此不能使用 SCB->VTOR 改变向量表基址。APP 中断必须通过 Bootloader 的硬件向量表和 SRAM 软向量表转发。

13 使用建议

1. 写外设驱动前，先用 SCU->PENR 打开对应外设时钟。
2. 修改外设配置前，必要时用 SCU->PRSTR 复位该外设。
3. APP 的 RAM 起始地址必须从 0x20000060 开始，避开软向量表。
4. Flash 写入必须限制在 APP 区：0x01002000-0x010077FF。
5. Flash 控制器地址 0x40008000 是寄存器区，不是代码 Flash 存储区。
6. GPIO 置位/清零/翻转优先使用 BSR/BRR/BFR，避免读改写竞争。
7. 4 KB SRAM 极小，应定期检查 map 文件，特别是 task stack、printf buffer 和日志 page buffer。